TU Wien
Faculty of Informatics
Research Unit Parallel Computing

Basics of Parallel Computing
2026S
Assignment 1

April 27, 2026

2 Person Group 1
1: Max SPANNRING, 12104980
2: Ida NEUHAUSER, 11806595

2 The Tasks

2.2 Compute Speed-up and Parallel Efficiency for 2 Instance Sizes

The following results were obtained when computing the speed-up and paralell efficiency for 2 instance sizes:

Table 1: Runtime and speed-up of parallel Julia set generator for c_s case.

size	p	mean runtime (s)	speed-up	par. eff.
120	1	0.1818	1.000	1.000
120	2	0.0983	1.850	0.925
120	4	0.0693	2.623	0.656
120	8	0.0528	3.444	0.431
120	16	0.0613	2.966	0.185
120	24	0.0729	2.496	0.104
120	32	0.0845	2.151	0.067
1030	1	12.3917	1.000	1.000
1030	2	6.1616	2.011	1.006
1030	4	3.1265	3.963	0.991
1030	8	1.5842	7.822	0.978
1030	16	0.8406	14.741	0.921
1030	24	0.6096	20.328	0.847
1030	32	0.5267	23.526	0.735

Tables 1 and 2 present the parallel runtime measurements and the respective speed-up and efficiency values for the cases c_s and c_b.

The following plots 1 to 6 visualize the results from tables 1 and 2:

Table 2: Runtime and speed-up of parallel Julia set generator for c_b case.

size	p	mean runtime (s)	speed-up	par. eff.
120	1	0.1692	1.000	1.000
120	2	0.0921	1.838	0.919
120	4	0.0609	2.778	0.694
120	8	0.0472	3.582	0.448
120	16	0.0544	3.108	0.194
120	24	0.0658	2.572	0.107
120	32	0.0777	2.178	0.068
1030	1	11.7121	1.000	1.000
1030	2	5.7853	2.024	1.012
1030	4	2.9386	3.986	0.996
1030	8	1.4778	7.925	0.991
1030	16	0.7890	14.845	0.928
1030	24	0.5766	20.311	0.846
1030	32	0.5062	23.139	0.723

Plots for $c = c_b$ (benchmark)

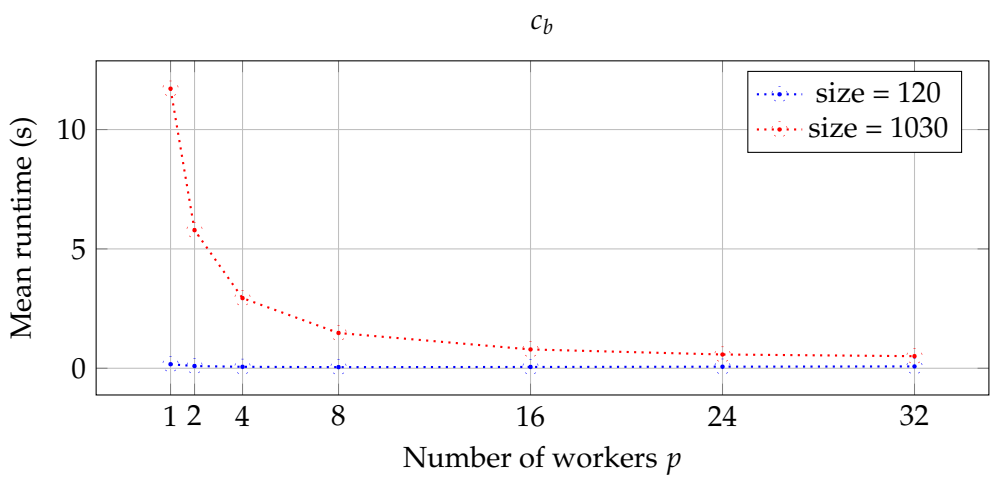

Figure 1: Mean runtime versus number of workers for $c = c_b$. As we can see, the mean runtime for large problem sizes is much larger at the beginning and thus dwarfes the graph for smaller runtimes. We can also implicitly see that the relative speedup is better for larger problem sizes.

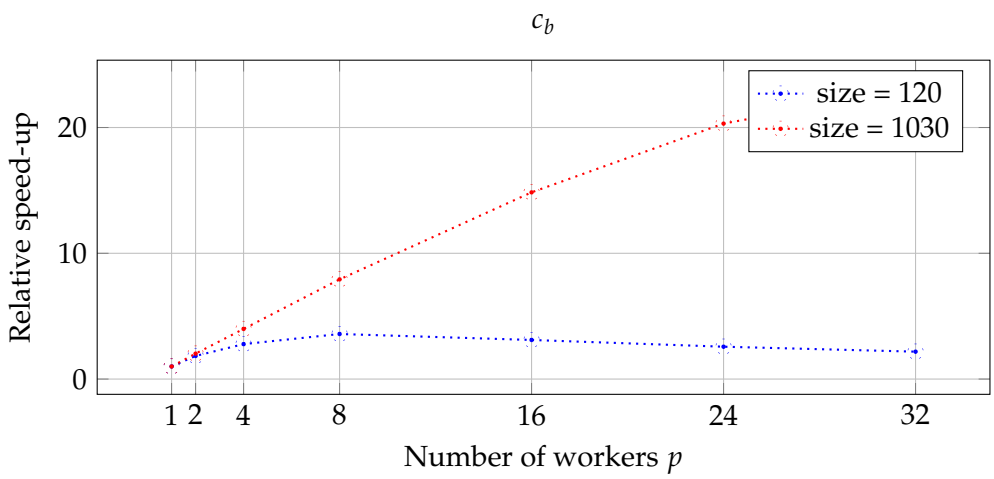

Figure 2: Relative speed-up versus number of workers for $c = c_b$. In the ideal case, we would expect a linear scaling $S(p) = p$. However, due to computational overhead, this is not the case. As already mentioned in the caption of figure 1, larger problem sizes have a better relative speed up because the overhead costs amortize.

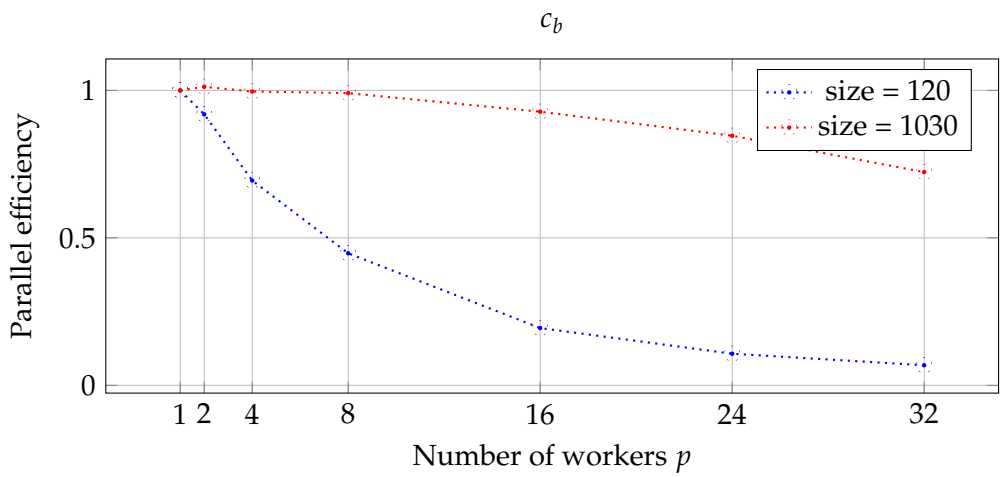

Figure 3: Parallel efficiency versus number of workers for $c = c_b$. The paralell efficiency of small problem sizes drops concerningly low very quickly. Large problem sizes are using more cores more efficiently.

Plots for $c = c_s$ (student)

Figure 4: Mean runtime versus number of workers for $c = c_s$. Similar to the case of $c = c_b$.

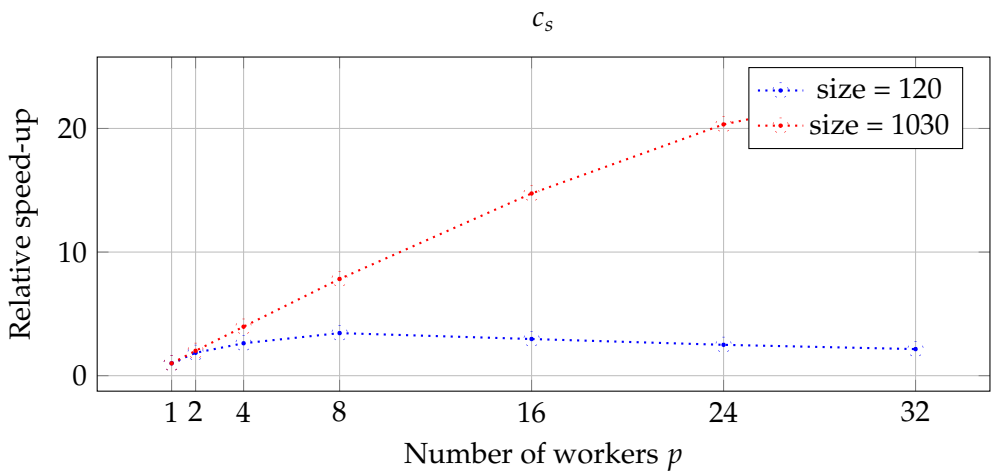

Figure 5: Relative speed-up versus number of workers for $c = c_s$. Similar to the case of $c = c_b$.

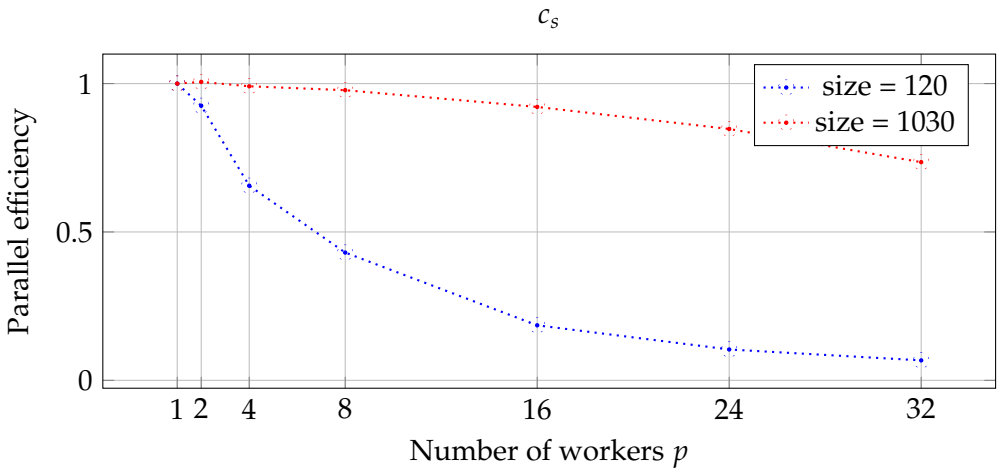

Figure 6: Parallel efficiency versus number of workers for $c = c_s$. Similar to the case of $c = c_b$.

Comparison of c_b and c_s

c_s and c_b deliver similar results. This is expected, since the calculation of c_s in `julia_curve.py` based on the group size and TUWEL number makes sure that the computational work per pixel is roughly comparable.

2.3 Influence of patch size

Table 3: Mean runtime as a function of patch size, with $size = 880$ and $p = 32$, three repetitions per configuration.

size	p	patch	mean runtime (s)
880	32	1	50.5364
880	32	5	2.1375
880	32	10	0.6247
880	32	20	0.4135
880	32	55	0.4658
880	32	150	1.1638
880	32	400	3.6837

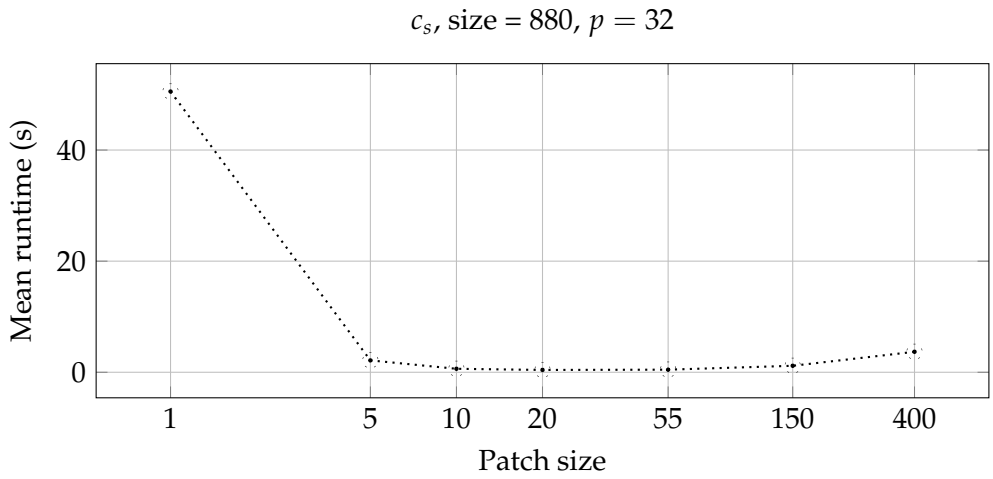

Figure 7: Mean runtime versus patch size for $c = c_s$, $size = 880$, $p = 32$. As we can see, the mean runtime drops significantly for patch sizes that are only a litle bit larger than 1. This is because very small patch sizes lead to large overhead. However, at the larger end for patch sizes we see an increase in mean runtime. This might be due to load balancing, since some workers finish earlier, because there are not enough patches for a comparatively small image.

2.4 Finding the best patch size

Table 4: Mean runtime as a function of patch size, with $size = 900$ and $p = 20$, three repetitions per configuration.

size	p	patch	mean runtime (s)
900	20	20	0.5431
900	20	21	0.5481
900	20	22	0.5438
900	20	23	0.5450
900	20	24	0.5491
900	20	25	0.5453
900	20	26	0.5435
900	20	27	0.5376
900	20	28	0.5422
900	20	29	0.5622
900	20	30	0.5452
900	20	31	0.5511
900	20	32	0.5459
900	20	33	0.5593
900	20	34	0.5446
900	20	35	0.5638
900	20	36	0.5677
900	20	37	0.5525
900	20	38	0.5707
900	20	39	0.5641
900	20	40	0.5697
900	20	41	0.5723
900	20	42	0.5819
900	20	43	0.5745
900	20	44	0.5678
900	20	45	0.5906
900	20	46	0.5805
900	20	47	0.5831
900	20	48	0.5776
900	20	49	0.5797
900	20	50	0.5892
900	20	51	0.6061
900	20	52	0.5796
900	20	53	0.5868
900	20	54	0.5715
900	20	55	0.6079
900	20	56	0.6065
900	20	57	0.6115
900	20	58	0.6090
900	20	59	0.5912
900	20	60	0.5873

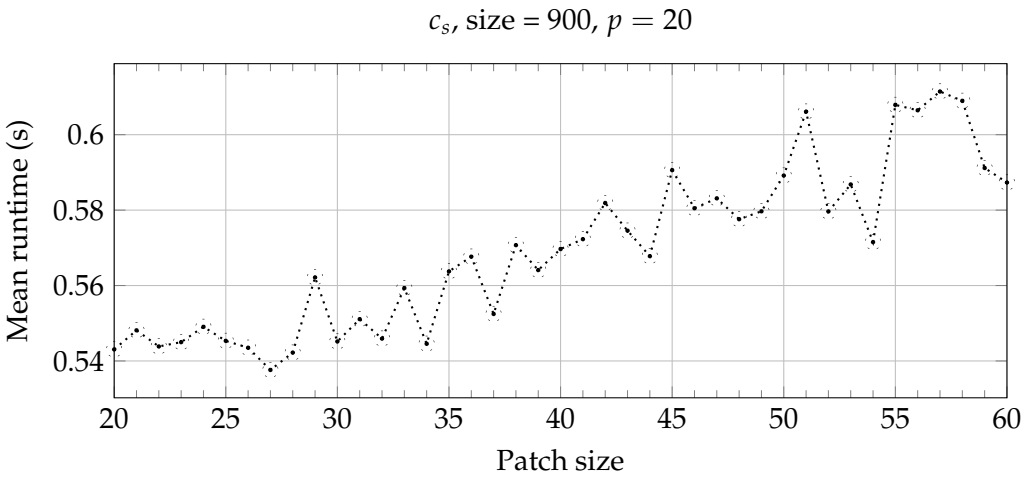

Figure 8: Mean runtime versus patch size for $c = c_s$, $size = 900$, $p = 20$, fine sweep over patch $\in [20, 60]$. The empirical analysis suggests that the ideal patch size is 27. However, the data seems quite noisy. Furthermore, the difference in the mean runtime is not significant.

3 Speed-up Analysis

Two algorithms A_1, A_2 are given with the following parallel runtimes:

$$T_{par}^{A_1}(n,p) \in O(\frac{n \log n}{p} + \log^2(n)),$$

$$T_{par}^{A_2}(n,p) \in O(\frac{n \log n}{p} + 3 \cdot n.$$

The sequential time for both is defined as $T_{seq}^{*}(n) \in O(n \log n)$.

The *absolute speed-up* S_a is given by $S_a(n,p) = \frac{T_{seq}(n)}{T_{par}(n,p)}$.
For algorithm A_1, this gives:

$$S_a^{A_1}(n,p) = \frac{n \log(n)}{\frac{n \log n}{p} + \log^2 n}.$$

As for algorithm A_2, this gives:

$$S_a^{A_2}(n,p) = \frac{n \log(n)}{\frac{n \log n}{p} + 3n}.$$

Table 5 shows the different *absolute speed-ups* for $n = 500, p = \{4, 16, 64\}$.

When comparing both algorithms with the ideal speed-up, A_1 performs nicely enough with 4 cores, moderately with 16 and bad with 64 cores. A_2 does not offer benefits of parallelization with any amount of cores. Figure 9 shows the relationship between ideal and absolute speed-ups.

Table 5: Absolute speed-up of algorithms with fixed n = 500 by number of cores.

p	4	16	64
$S_a^{A_1}$	3.81	13.35	35.65
$S_a^{A_2}$	1.36	1.83	2.00

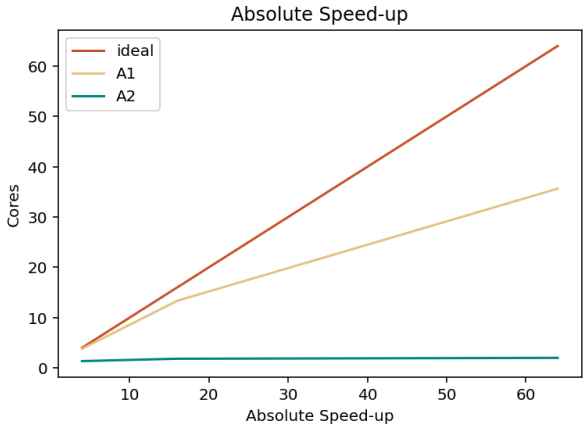

Figure 9: Absolute Speed-up by cores and algorithms.

The absolute speed-up alone does not quantify the gain from parallelizing a code. If we look at the parallel efficiency is given by $E(n,p) = \frac{T_{seq(n)}}{p \cdot T_{par}(n,p)} = \frac{S_a(n,p)}{p}$, we can see that for both algorithms, the efficiency drops off quite quickly (see Figure 10). Table 6 shows the parallel efficiency of the algorithms. Both do not scale well.

Table 6: Parallel efficiency of algorithms with fixed n = 500 by number of cores.

p	4	16	64
$S_a^{A_1}$	0.95	0.83	0.56
$S_a^{A_2}$	0.34	0.11	0.03

The potential speed-up is determined by the sequential fraction of a code.

For A_1, the sequential fraction s_{A1} is given by:

$$s_{A1}(n) = \frac{\log^2(n)}{n \log n} = \frac{\log n}{n}.$$

The potential speed-up for algorithm 1 is then given by

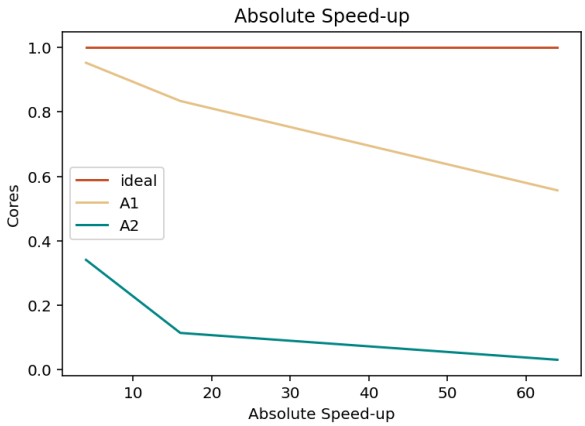

Figure 10: Absolute Speed-up by cores and algorithms.

$$S^{A_1}(n,p) = \frac{1}{s_{A1} + \frac{1-s_{A1}}{p}} = \frac{1}{\frac{\log n}{n} + \frac{1-\frac{\log n}{n}}{p}} \leq \frac{n}{\log n} = S^{A_1}(n).$$

Regarding A_2, s_{A2} is given by:

$$s_{A2}(n) = \frac{3 \cdot n}{n \log n} = \frac{3}{\log n}.$$

The potential speed-up for algorithm 2 is then given by

$$S^{A_2}(n,p) = \frac{1}{s_{A2} + \frac{1-s_{A2}}{p}} = \frac{1}{\frac{3}{\log n} + \frac{1-\frac{3}{\log n}}{p}} \leq \frac{\log n}{3} = S^{A_2}(n).$$

4 Weak Scaling Analysis